Extended Game Theory Notes

from Mathematical Optimization and Economic Theory

Jacob Wyngaard

Contents

1 Finite Two-Player Games and Nash Equilibrium

1.1 Basic setup

Consider a finite two-player normal-form game. Player 1 has m pure strategies, Player 2 has n pure strategies.

- Player 1 payoff matrix: $A \in \mathbb{R}^{m \times n}$
- Player 2 payoff matrix: $B \in \mathbb{R}^{m \times n}$

A mixed strategy for Player 1 is a probability vector

$$p^1 \in \{x^\top \in \mathbb{R}^m : x \geq 0,\ x\mathbf{1} = 1\},$$

and similarly Player 2 chooses

$$p^2 \in \{y^\top \in \mathbb{R}^n : y \geq 0,\ y\mathbf{1} = 1\}.$$

Expected payoffs:

$$u_1(p^1, p^2) = p^1 A (p^2)^\top, \qquad u_2(p^1, p^2) = p^1 B (p^2)^\top.$$

Definition. A pair $(p^{1*}, p^{2*}) \in R^m \times R^n$ is a **Nash equilibrium** if

$$p^{1*} \in \arg\max_{p^1 \in R^m} p^1 A (p^{2*})^\top \quad \text{and} \quad p^{2*} \in \arg\max_{p^2 \in R^n} p^{1*} B (p^2)^\top.$$

Equivalently, (p^{1*}, p^{2*}) is a Nash equilibrium iff no player can improve by unilateral deviation:

$$p^1 A (p^{2*})^\top \leq p^{1*} A (p^{2*})^\top \quad \forall p^1 \in R^m, \qquad p^{1*} B (p^2)^\top \leq p^{1*} B (p^{2*})^\top \quad \forall p^2 \in R^n.$$

1.2 Supports and indifference

Let $\mathrm{supp}(p) := \{i : p_i > 0\}$. At a Nash equilibrium:

- All pure strategies in $\mathrm{supp}(p^{1*})$ give Player 1 the same payoff against p^{2*}.
- Any pure strategy outside the support does *not* yield a higher payoff.

Formally, define the pure payoff vectors against p^2:

$$(A(p^2)^\top)_i = e_i^\top A (p^2)^\top.$$

Then at equilibrium there exists a scalar v_1 (Player 1's equilibrium payoff) such that

$$(A(p^{2*})^\top)_i = \begin{cases} v_1 & i \in \mathrm{supp}(p^{1*})), \\ \leq v_1 & i \notin \mathrm{supp}(p^{1*}). \end{cases}$$

Similarly, letting $(B^\top x)_j = x^\top B e_j$, there exists v_2 such that

$$(p^{1*} B)_j = \begin{cases} v_2 & j \in \mathrm{supp}(p^{2*})), \\ \leq v_2 & j \notin \mathrm{supp}(p^{2*})). \end{cases}$$

This implies that if $(A(p^{2*})^\top)_i < v_1$, then Player 1 chooses strategy i with probability 0. Likewise, if $(p^{1*}B)_j < v_2$, then Player 2 chooses strategy j with probability 0. This is analogous to the Strong Minimax Theorem in zero-sum games.

2 Zero-Sum Games and the Minimax Theorem

A game is **zero-sum** if Player 2's payoff is the negative of Player 1's payoff:

$$B = -A.$$

Then $u_2(p^1, p^2) = -u_1(p^1, p^2)$ and we focus on the value of the game.

Definition. The **value** of a zero-sum game with payoff matrix A is

$$v := \max_{p^1 \in R^m} \min_{p^2 \in R^n} p^1 A (p^2)^\top = \min_{p^2 \in R^n} \max_{p^1 \in R^m} p^1 A (p^2)^\top.$$

Theorem. (**von Neumann Minimax Theorem**) For finite zero-sum games,

$$\max_{p^1 \in R^m} \min_{p^2 \in R^n} p^1 A (p^2)^\top = \min_{p^2 \in R^n} \max_{p^1 \in R^m} p^1 A (p^2)^\top.$$

Moreover, there exists an equilibrium (p^{1*}, p^{2*}) such that p^{1*} and p^{2*} achieve these extrema.

3 Dominance and Elimination (Including Mixed Dominance)

3.1 Strict dominance by pure strategies

Player 2 pure strategy j is **strictly dominated** by pure strategy k (for Player 2) in a zero-sum game if it gives Player 2 a strictly worse payoff against every Player 1 action, i.e.

$$B_{ij} < B_{ik} \quad \forall i.$$

Since Player 2 maximizes their payoff, Player 2 would never play j.

3.2 Strict dominance by a mixed strategy

More generally, a column j is strictly dominated by a *mixed* strategy over other columns if there exists $w \in R^n$ with $w \geq 0$ such that,

$$B_{ij} < \sum_{\ell \neq j} w_\ell B_{i\ell} \quad \forall i, \qquad \sum_{\ell \neq j} w_\ell = 1$$

In this case, $\sum_{\ell \neq j} w_\ell B_{i\ell}$ is a convex hull of Player 2 strategies other than j

3.3 Row minima obstruction

If a Player 2 column j attains a *row minimum* in some row i (i.e. $A_{ij} = \min_k A_{ik}$), then it cannot be strictly dominated by any convex combination of other columns.

Proof

Suppose A_{ij} is minimal in row i

For any convex combination $\sum_{\ell \neq j} w_\ell A_{i\ell}$,

we have $\sum_{\ell \neq j} w_\ell A_{i\ell} \geq \min_{\ell \neq j} A_{i\ell} \sum_{\ell \neq j} w_\ell \geq \min_{\ell \neq j} A_{i\ell} \geq A_{ij}$

Thus, strict domination ($A_{ij} > \sum w_\ell A_{i\ell}$) fails in that row and therefore fails overall

4 Support Enumeration

4.1 Idea

Support enumeration searches over candidate supports and solves the induced *indifference equations*. Let $I \subseteq \{1, \ldots, m\}$ and $J \subseteq \{1, \ldots, n\}$ be candidate supports with $|I| = |J| = k$, where $k \in \{1, \ldots, \min(m, n)\}$. Write p_I^1 and p_J^2 for the subvectors, and $A_{I,J}$, $B_{I,J}$ for the submatrices.

4.2 Indifference equations

We require:

$$A_{I,J}(p_J^2)^\top = \lambda \mathbf{1}, \qquad p_I^1 B_{I,J} = \mu \mathbf{1},$$

together with normalization constraints:

$$p_I^1 \mathbf{1} = 1, p_J^2 \mathbf{1} = 1,$$

These are two sets of linear equations with $k + 1$ unknowns and $k + 1$ constraints.

4.3 Feasibility and best-response checks

After solving:

- **Positivity:** $x_I \geq 0$, $y_J \geq 0$.
- **No profitable deviations:**

$$(A(p^2)^\top)_i \leq \lambda \quad \forall i \notin I, \qquad (p_1 B)_j \leq \mu \quad \forall j \notin J.$$

Algorithm. (**Support Enumeration**)

1. For each candidate support pair (I, J):
2. Solve the indifference + normalization system for $(p_I^1, p_J^2, \lambda, \mu)$.
3. If p_I^1, p_J^2 are feasible probabilities and deviation inequalities hold, record (p^1, p^2) as a Nash equilibrium.

5 Vertex Enumeration via Best-Response Polytopes

5.1 Complementarity conditions and fully labeled vertices

Vertex enumeration methods represent equilibria as *fully labeled* vertex pairs of two polytopes (one per player). This method introduces nonnegative vectors that encode the complementary slackness constraints. One common formulation (up to scaling conventions) is:

Definition. Define the best-response polytopes

$$P := \{p^1 \in \mathbb{R}^m : p^1 \geq 0,\ p^1\mathbf{1} = 1,\ p^1 B \leq u\mathbf{1}\},$$

$$Q := \{p^2 \in \mathbb{R}^n : p^2 \geq 0,\ p^2\mathbf{1} = 1,\ Ap^2 \leq v\mathbf{1}\},$$

Here u and v represent equilibrium expected payoffs bounds.

5.2 Labeling interpretation

Each inequality (or nonnegativity constraint) can be given a label. A vertex is **fully labeled** if across the pair of vertices (one in P and one in Q) every label appears.

Intuitively:

- Labels from $(p^1)_i \geq 0$ correspond to whether Player 1 strategy i is used ($(p^1)_i > 0$) or excluded ($(p^1)_i = 0$).
- Labels from $(A(p^2)^\top)_i \leq v$ correspond to whether Player 1 strategy i is a best response (tight) or not (slack).

The same applies symmetrically for Player 2.

Theorem. A pair of points $p^1 \in P$ and $p^2 \in Q$ corresponds to a Nash equilibrium (p^1, p^2) if and only if the pair is fully labeled (equivalently, satisfies the complementarity conditions).

5.3 Why vertex enumeration is different from simplex

Simplex is optimized to find *one* optimal vertex of *one* polytope for a chosen linear objective. Vertex enumeration seeks to systematically generate vertices (and then match fully labeled pairs) across *two* interacting polytopes.

6 Computing Vertices: Geometric Duality Workflow

6.1 Primal: halfspace / facet representation

A polytope in $\mathbb{R}^d$ can be described as an intersection of halfspaces:

$$\mathcal{P} = \bigcap_{i=1}^{M} \{x \in \mathbb{R}^d : a_i^\top x + b_i \leq 0\}.$$

A facet (supporting hyperplane) is

$$a_i^\top x + b_i \le 0.$$

6.2 Shift to an interior point

Choose an interior point $x_0 \in \text{int}(\mathcal{P})$ and shift coordinates:

$$z := x - x_0.$$

Then facets become

$$a_i^\top (z + x_0) + b_i \le 0 \quad \Longleftrightarrow \quad a_i^\top z + \beta_i \le 0 \quad \text{where } \beta_i = b_i + a_i^\top x_0$$

If x_0 is in the interior, then $\beta_i < 0$ for all facets.

6.3 Facet-to-vertex dual mapping

A standard polarity-like mapping sends each shifted facet

$$a_i^\top z + \beta_i \le 0 \quad (\beta_i < 0)$$

to a dual vertex

$$w_i := \frac{a_i}{-\beta_i}.$$

Then the convex hull of $\{w_i\}$ encodes the dual polytope, and faces/vertices swap roles:

$$\text{(facets in primal)} \leftrightarrow \text{(vertices in dual)}, \qquad \text{(vertices in primal)} \leftrightarrow \text{(facets in dual)}.$$

6.4 Pipeline summary

Algorithm. (**Duality pipeline for vertex computation**)

1. Find an interior point x_0 and shift $z = x - x_0$.
2. Convert primal facets $a_i^\top z + \beta_i \le 0$ to dual vertices $w_i = a_i / -\beta_i$.
3. Compute facets of the dual polytope $\text{conv}\{w_i\}$ (e.g. via convex hull).
4. Convert those dual facets back into primal vertices.
5. Shift back: $x = z + x_0$.

Remark. In game-theory applications, after computing raw vectors you often renormalize them into probability vectors:

$$p \leftarrow \frac{p}{\mathbf{1}^\top p} \quad (\text{assuming } \mathbf{1}^\top p > 0).$$

7 Simplex Algorithm (Optimization on One Polytope)

7.1 Standard form and slack variables

A linear program in inequality form:

$$\max\ c^\top x \quad \text{s.t.} \quad Ax \le b,\ x \ge 0$$

is converted to standard form by introducing slack variables $s \ge 0$:

$$Ax + s = b, \qquad x \ge 0,\ s \ge 0.$$

7.2 Basic feasible solutions and pivots

Let the variables be partitioned into basic and nonbasic sets. A **basic feasible solution** sets nonbasic variables to 0 and solves for basic variables.

At each iteration, the algorithm:

1. Computes **reduced costs** to see whether increasing a nonbasic variable improves the objective.
2. Picks an entering variable (improving direction).
3. Uses the **minimum ratio test** to maintain feasibility and choose a leaving variable.
4. Performs a **pivot** to move to an adjacent vertex along an edge.

> **Theorem.** If all reduced costs are ≤ 0 (for a maximization problem), the current basic feasible solution is optimal.

7.3 Worked style

Given constraints of the form

$$\begin{cases} a_1^\top x \le b_1 \\ \vdots \\ a_M^\top x \le b_M \end{cases}$$

introduce slacks $s_i \ge 0$:

$$a_i^\top x + s_i = b_i.$$

At a vertex, a subset of constraints are tight, which corresponds to certain slacks being 0. This is why nonbasic variables are 0 at that vertex.

8 Cooperative Games with Side Payments

8.1 Characteristic function and imputations

Let $N = \{1, \dots, n\}$ be players. A **coalitional game** is given by a value function

$$v : 2^N \to \mathbb{R}, \qquad v(\emptyset) = 0,$$

where $v(S)$ is the total transferable payoff available to coalition S.

Note that v is **super-additive**: $v(A \cup B) \geq v(A) + v(B) \quad \forall A, B \in 2^N$ satisfying $A \cap B = \emptyset$

Definition. An **imputation** is a payoff vector $x \in \mathbb{R}^n$ satisfying:

- **Individual rationality**: $x_i \geq v(\{i\})$ for all $i \in \{1, ..., n\}$.
- **Group rationality**: $\sum_{i=1}^{n} x_i = v(\{1, ..., n\})$.

8.2 The core

Definition. An imputation x is **blocked** by coalition $S \subseteq \{1, ..., n\}$ if there exists $y \in \mathbb{R}^{|S|}$ such that

$$\sum_{i \in S} y_i = v(S) \quad \text{and} \quad y_i > x_i \ \forall i \in S.$$

In this case, a sub-coalition of players benefits by splitting off from the main group and finding their own imputation. The **core** is the set of imputations not blocked by any coalition.

Equivalent inequality description of the core:

$$\text{Core}(v) = \left\{ x \in \mathbb{R}^n : \sum_{i \in N} x_i = v(N), \ \sum_{i \in S} x_i \geq v(S) \ \forall S \subseteq N \right\}.$$

Each possible sub-coalition enforces another constraint on the system's imputations.

8.3 Shapley value

Definition. The **Shapley value** $\phi(v) \in \mathbb{R}^n$ is given by

$$\phi_i(v) = \sum_{S \subseteq N \setminus \{i\}} \frac{|S|!(n - |S| - 1)!}{n!} \big(v(S \cup \{i\}) - v(S) \big).$$

Interpretation: $v(S \cup \{i\}) - v(S)$ is player i's marginal contribution to coalition S, and the coefficient is the probability that S precedes i in a uniformly random ordering of players.

8.4 Negligible players and limiting behavior

In large games with "many small" (negligible) players, Shapley-type allocations often approximate competitive outcomes.

Appendix A: Support Enumeration Python Program

```
import numpy as np
from itertools import combinations
import math
from fractions import Fraction

def manual_support_enumeration(A,B):
    m, n = A.shape
    max_support = min(m,n)
    for k in range(1,max_support+1):
        player_1_supports = list(combinations(range(m),k))
        player_2_supports = list(combinations(range(n),k))

        num_S1s = math.comb(m,k)
        num_S2s = math.comb(n,k)

        for index in range(num_S1s*num_S2s):

            S1_index = index % num_S1s
            S2_index = index // num_S1s

            S1 = player_1_supports[S1_index]
            S2 = player_2_supports[S2_index]

            reduced_A = A[np.ix_(S1, S2)]
            reduced_B = B[np.ix_(S1, S2)]

            # Solve Indifference Equations

            # [ reduced_B^T -1 ] [p^1]  = [0]
            # [ 1^T          0 ] [mu]     [1]

            ones = np.ones(k)
            matrix_1 = np.block([[reduced_B.T,-ones[:, None]],
                [ones[None, :], np.zeros((1, 1))]])

            # [ reduced_A   -1 ] [p^2]  = [0]
            # [ 1^T          0 ] [lambda]     [1]

            matrix_2 = np.block([[reduced_A,-ones[:, None]],
                [ones[None, :], np.zeros((1, 1))]])

            vector = np.concatenate([np.zeros(k), np.array([1.0])])

            try:
                solution_1 = np.linalg.solve(matrix_1, vector)
                solution_2 = np.linalg.solve(matrix_2, vector)
            except np.linalg.LinAlgError:
                continue

            p1 = np.zeros(m)
            p2 = np.zeros(n)
```

```
52
53              for index in range(k):
54                  p1[S1[index]] = solution_1[index]
55                  p2[S2[index]] = solution_2[index]
56
57              mu = solution_1[k]
58              lam = solution_2[k]
59
60              # Check for Nash Equilibrium
61              nash_equilibrium = np.all(p1>=-1e-10) and np.all(p2>=-1e-10)
62
63              if nash_equilibrium:
64                  expected_values_1 = A@p2
65                  for strategy in [index for index in range(m) if index not in set(S1)]:
66                      if expected_values_1[strategy] > lam + 1e-10:
67                          nash_equilibrium = 0
68                          break
69
70              if nash_equilibrium:
71                  expected_values_2 = p1@B
72                  for strategy in [index for index in range(n) if index not in set(S2)]:
73                      if expected_values_2[strategy] > mu + 1e-10:
74                          nash_equilibrium = 0
75                          break
76
77              if nash_equilibrium:
78                  print("Nash Equilibrium!")
79                  p1 = [Fraction(x).limit_denominator() for x in p1]
80                  p2 = [Fraction(x).limit_denominator() for x in p2]
81                  print("p1: ",p1)
82                  print("p2: ",p2)
83      return
```